

Lecture 7: Reinforcement Learning

Satchit Chatterji

SIAS Group, Ivi, Universiteit van Amsterdam

(based on a slideset by Jelle Zuidema & Iris Proff)

Machine Learning and Language Models, 2026

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Types of learning algorithms

Supervised learning

Signal: labeled data (x, y)

Learn: a predictive mapping

$$\hat{y} = f(x) \approx y$$

Examples: image recognition, sentiment analysis, spam detection...

Unsupervised learning

Signal: unlabeled data x

Learn: structure or representation

$$z = g(x)$$

Examples: document clustering, dimensionality reduction, anomaly detection...

Reinforcement learning

Signal: rewards from interaction

Learn: values or a policy

$Q(s, a) \approx$ future reward,
 $\pi(a | s)$ chooses actions.

Examples: robot navigation, game playing, dialogue systems...

Reinforcement learning

A structural model from behavioral psychology...

Pavlov (1897) Classical conditioning

Learn a stimulus–outcome contingency

Thorndike (1905) Law of effect / instrumental conditioning

Learn an action–outcome contingency

Tolman (1930) Latent / goal-directed learning

Learn a cognitive map of the environment

Common idea:

Experience changes future behavior according to its consequences.

... formalized by AI as computational models

Behavioral description

Cognitive / AI model

Classical conditioning

→ Rescorla–Wagner model (1972)

Instrumental conditioning

→ Temporal-difference learning (Samuel, 1959)

Q-learning (Watkins & Dayan, 1992)

Goal-directed learning

→ Model-based learning

$$U(S) = R(S)$$

*State-transition diagram
from source slide 10*

*The value of a state depends on the
reward it provides.*

*State-transition diagram from source slide
11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount factor

State-transition diagram from source slide
11

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount factor

Question 1

Consider states S_2 and S_3 (which can you reach with action A_1 and A_2) and state S_4 (which you can reach from state S_3 by action A_1). How valuable are each of these states relative to each other? Is $U(S_1) = U(S_3)$?

*State-transition diagram from source slide
11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount factor

*State-transition diagram from source slide
11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount factor

Question 2

How do you avoid having to first learn everything about the environment/problem before you can start acting in the world?

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Q-learning and temporal-difference learning

How do we learn the utility of state–action pairs by trial and error?

Learning the Q-value

$$Q(S, A) = \hat{U}(S, A)$$

Q-learning approximates utility through online learning: the policy is applied and learned at the same time.

Q-learning update rule

Prediction error

$$\delta = R + \gamma \max_{A'} Q_t(S', A') - Q_t(S, A)$$

Q-value update

$$Q_{t+1}(S, A) = Q_t(S, A) + \alpha \delta$$

Asymptotic relationship

$$\lim_{t \rightarrow \infty} Q_t(S, A) = U(S, A)$$

Reminder: $U(S, A) = R(S, A) + \gamma \max_{A'} [U(S', A')]$.

State-transition diagram from source slide 13

Q-learning action-selection policy

Suppose we are in state S , with possible actions A_1 and A_2 , and

$$Q(S, A_1) = 0.5, \quad Q(S, A_2) = 0.7.$$

Which action should we choose?

Q-learning action-selection policy

Suppose we are in state S , with possible actions A_1 and A_2 , and

$$Q(S, A_1) = 0.5, \quad Q(S, A_2) = 0.7.$$

Which action should we choose?

Question 3

How do you make sure you maximize reward, both in the short term (taking advantage of your knowledge) and in the long term (extending your knowledge)?

Exploration–exploitation trade-off

Exploit what you know

or

Explore new options

An agent has to exploit what it has experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

Sutton & Barto (2018)

Exploration–exploitation trade-off

Exploit what you know

or

Explore new options

An agent has to exploit what it has experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

Sutton & Barto (2018) Classic example: [multi-armed bandit](#)

Multi-armed bandit



The name comes from imagining a gambler at a row of slot machines ("one-armed bandits"), who has to decide which machines to play, how many times to play each machine and in which order to play them, and whether to continue with the current machine or try a different machine.

Q-learning action-selection policies

Suppose $Q(S, A_1) = 0.5$ and $Q(S, A_2) = 0.7$. How should we trade off exploration and exploitation?

Epsilon-greedy

$$P(a_i | S) = \begin{cases} 1 - \epsilon & \text{if } q_i = \max_A q_i(A), \\ \frac{\epsilon}{|A| - \max_A q_i(A)} & \text{otherwise.} \end{cases}$$

The best action is chosen with probability $1 - \epsilon$.

Softmax

$$P(a_i | S) = \frac{e^{q_i/\tau}}{\sum_A e^{q_i/\tau}}$$

For $\tau \rightarrow 0$ the policy is greedy; for $\tau \rightarrow \infty$ it is random.

Q-learning pseudo-code

1. Set $\alpha \in [0, 1]$, small $\epsilon > 0$, and $\gamma > 0$.
2. Initialize $Q(s, a)$ for all $s \in S$ and $a \in A$ to 0.
3. Initialize s .
4. For each episode:
 - 4.1 While s is not terminal:
 - 4.1.1 Choose a from s using the action-selection policy.
 - 4.1.2 Observe reward R and next state s' .
 - 4.1.3 Update Q and set $s \leftarrow s'$.

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Temporal-difference learning in the brain

*Dopaminergic-neuron recording chart from
source slide 18*

- Recording from a single dopaminergic neuron in a monkey brain
- Upper blue: expected reward
- Red: expected reward omitted
- Green: unexpected early or late reward
- Lower blue: expected reward

Hollerman (1998)

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Model-based versus model-free learning

Model-free (habitual)

Rely on trial and error. Efficient, but inflexible.

Model-based (goal-directed)

Build a model of the environment. Effortful, but flexible.

Dayan & Niv (2008)

Environment model

$$M : (s, a) \mapsto (s', r)$$

What does my environment look like?

State-transition diagram from source slide 20

Sutton & Barto (2018)

1. Initialize parameters.
2. For each episode, while s is not terminal:
 - 2.1 Choose a from s using the action-selection policy.
 - 2.2 Observe R and s' , update Q , and set $s \leftarrow s'$.
 - 2.3 Update the model: $M(s, a) = (R, s')$.
 - 2.4 Repeat n times:
 - 2.4.1 Choose (s, a) to simulate.
 - 2.4.2 Simulate the action and observe R and s' .
 - 2.4.3 Update Q .

Direct RL update and model learning use real experience; planning uses simulated experience.

Policy during episode 2

Dyna-Q policy diagram from source slide 22

Sutton & Barto (2018)

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

- **Eligibility traces:** update not only the last state–action pair, but all previously visited pairs with a decay factor λ .
- **Decaying learning rate:** α decreases over time.
- **Hybrid models:** combine model-based and model-free components.

What is Reinforcement Learning?

Q-learning

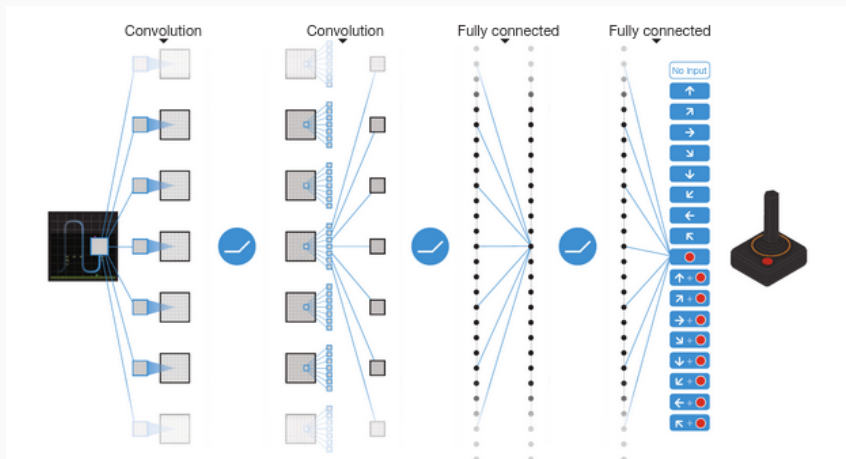
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Deep Reinforcement Learning



Deep Q-Learning (Mnih et al. 2015)

- Replace Q-table with a neural network Q , with weights θ ; use it to compute Q-value of current state
- Make a copy of the Q-network (the *target-network* $\hat{Q}$); use it to compute Q-value of target states
- Learn in batches: act in the world (choose actions using ϵ -greedy) and gather data
- Optimize Q-network (minimize prediction error) via gradient descent on a batch of observations
- Occasionally copy Q into $\hat{Q}$

Deep Q-Learning (Mnih et al. 2015)

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

End For

LETTER

Human-level control through deep reinforcement learning

Vedajitpryash Mnih*, Koray Kavukcuoglu*, David Silver†, Andrej A. Ruess†, Joel Veness*, Marc G. Bellemare*, Alex Graves*, Martin Riedmiller*, Andrius K. Pajula*, Georg Ostrovski*, Sjoerd Woutman*, Charles Beattie*, Arav Siddh*, Iqbal Anwar*, Helen King*, Jonathan Schritter*, Dean Whalley*, Shane Legg & Demis Hassel*

The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimise their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalise past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems¹, the former evidenced by a wealth of neural data revealing useful parallels between the phasic signals emitted by dopaminergic neurons and temporal-difference reinforcement learning algorithms². While reinforcement learning agents have achieved some success in a variety of domains^{3–5}, their applicability has previously been limited to domains in which useful features can be hand-crafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks^{6–14} to develop a novel architecture for learning a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games¹⁵. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human player across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This result bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to control at a human level of challenging tasks. We set out to create a single algorithm that is able to learn a wide range of competencies on a varied range of challenging tasks: a central goal of general artificial intelligence¹⁶ that has eluded previous efforts^{17–19}. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural networks²⁰ known as deep neural networks. Notably, recent advances in deep neural networks^{21–23}, in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional network²⁴, which uses hierarchical layers of efficient convolutional filters to extract the effects of receptive fields and the power of ReLU²⁵ as a universal non-linear foundation processing in early visual cortex²⁶—thereby exploiting the local spatial correlations present in images, and building in robustness to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the

agent is to select actions in a fashion that maximises cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_{a'} \{r(s,a') + \gamma V(s') + \dots\}, \quad (s,a) \in \mathcal{S} \times \mathcal{A},$$

which is the maximum sum of rewards, discounted by γ at each time step, achievable by a behaviour policy $\pi = \{ \pi(a|s) \}$, after making an observation (s) and taking an action (a) (see Methods²⁷).

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function²⁸. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy, and therefore change the data distribution, and the correlations between feature values ($Q(s)$ into feature values $v = \pi(s)$ ($Q(s), v$)). We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we build a biologically inspired mechanism termed experience replay²⁹, that randomises over the data, thereby removing correlations in the observation sequence and ensuring other changes in the data distribution (see below) do not affect the learning. We use an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other methods exist for learning neural networks in the reinforcement learning setting, such as neural Q-learning³⁰, these methods involve the repeated training of networks to solve thousands of instances. Consequently, these methods, unlike our algorithm, are not sufficient to be used successfully with large neural networks. We parameterise an approximate value function ($Q(s,a)$) using the deep convolutional neural network shown in Fig. 1, in which W are the parameters (that is, weights) of the Q-network at iteration t . To perform experience replay, we store the agent's experience $\mathcal{D} = \{ (s_t, a_t, r_t, s_{t+1}) \}$ at each time step t in a data set $\mathcal{D} = \{ s_1, \dots, s_n \}$. During learning, we apply Q-learning updates on samples (reminiscent of mini-batches) $(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}$. DQN differs uniformly at random from the pool of stored samples. The Q-learning update at iteration t uses the following loss function

$$L_t(W) = \frac{1}{2N} \sum_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right)^2 \right]$$

in which γ is the discount factor determining the agent's horizon. A are the parameters of the Q-network at iteration t and Q' are the network's parameters used to compute the target at iteration t . The target network parameters Q' are updated by the Q-network parameters Q every C steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ($n = 49$) designed to be

NEWS & VIEWS

EDITORIAL INTELLIGENCE

Learning to see and act

An artificial intelligence system uses machine learning from massive training sets to teach itself to play 49 classic computer games, demonstrating that it can adapt to a variety of tasks. See Letter p.529

BERNARD SCHÖDLER

Improvements in our ability to process huge amounts of data have led to progress in many areas of science, not least artificial intelligence (AI). With advances in machine learning, less clever the development of machines that can learn intelligent behaviour directly from data, rather than being explicitly programmed to exhibit such behaviours. For instance, the advent of 'big data' has resulted in systems that can recognise objects or words with considerable precision. On page 529 of this issue, Mnih et al. describe an agent that uses large data sets to teach itself how to play 49 classic Atari 2600 computer games by looking at the pixels and learning actions that increase the game score. If that a professional player to many tasks — a remarkable achievement of the progress being made in AI. In machine learning, systems are trained to solve patterns from observational data. A particularly simple type of pattern, a mapping between input and output, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

model to explain these data (a process called function approximation). It does this by choosing from a class of model specified by the system designer. Designing this class is an art: its size and complexity should reflect the amount of training data available, and its content should reflect prior knowledge that the designer of the system considers useful for the problem at hand. If this is done well, the inferred model will then apply not only for the training set, but also for other data that adhere to the same underlying pattern. The rapid growth of data sets means that machine learning can now use complex, model classes and include highly non-trivial tasks. Such problems are usually characterised by several factors: the data are unlabelled; the underlying pattern is complex (for instance, it might be non-linear or changeable); and the designer has only weak prior knowledge about the problem — in particular, a mechanistic understanding is lacking. The human brain is typically robust to such problems: using input and output, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

we often learn without a 'supervisor' telling us the output of a hypothetical input-output function. Here, 'reinforcement' has a central role in learning behaviour from weak supervision. Machine learning adopted this idea to develop reinforcement learning algorithms, in which supervision takes the form of a numerical reward signal³¹, and the goal is for the system to learn a policy that, given the current state, determines which actions to pick to maximise an accumulated future reward. Mnih et al. use a form of reinforcement learning known as Q-learning³² to teach systems to play a set of 49 training video games, learning how to increase the game score as a numerical reward. In Q-learning Q(s,a) represents the accumulated future reward, Q' if at state s the system first performs action a , and subsequently follows an optimal policy. The system tries to approximate Q' by using an artificial neural network — a function approximator loosely inspired by biological neural networks — called a deep Q-network (DQN). The DQN's input (the pixels from four consecutive game screens) is processed by connected 'hidden' layers of computations, which extract more and more specialised neural features to help approximate the complex

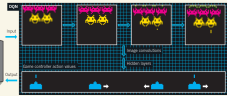


Figure 1 | Computer games. Mnih et al. have designed an artificial intelligence system, using a deep Q-network (DQN), that learns how to play 49 video games. The DQN analyses a sequence of four game screens (displays) and approximates, for each possible action a , the value of the consequences in the future game score r that action will bring and followed by the best possible course of subsequent actions. The first term of the DQN

analyses the pixels of the game screen and extract information from state and move-provided visual features (image coordinates). Subsequent fully connected hidden layers predict the value of actions from these features. The last layer is the output — the action taken by the DQN. The possible outputs depend on the specific game the system is playing; everything else is the state to which the DQN

*Google Research, 1 New Street Square, London EC4A 3DF, UK.
†Google Academic Research, 1 New Street Square, London EC4A 3DF, UK.

Deep Reinforcement Learning

- Deep reinforcement learning
- Learn Q-values with deep neural networks

A tutorial on deep reinforcement learning

<https://wiki.pathmind.com/deep-reinforcement-learning>